🎓 Purdue CS CODO Eligibility Advisor

An AI-powered Retrieval-Augmented Generation (RAG) application that helps Purdue University students determine their eligibility to Change their Degree Objective (CODO) into the Computer Science major. Students anonymize and upload their transcript, then chat with an AI advisor backed by official CODO policy documents.

Architecture Overview

```
┌──────────────────────────────────┐
│          React Frontend          │
│   (Vite · localhost:5173)        │
│                                  │
│   ┌──────────┐   ┌──────────────┐ │
│   │Anonymize│   │    Chat      │ │
│   │  Tab    │──▶│    Tab       │ │
│   └──────────┘   └──────────────┘ │
└──────────────────────┬──────────┘
                       │ HTTP (fetch)
                       ▼
┌──────────────────────────────────┐
│        FastAPI Backend           │
│   (Uvicorn · localhost:8000)     │
│                                  │
│   POST /chat    POST /ingest     │
│   GET  /status                   │
└──────────────────────────────────┘
                       │
          ┌────────────┼────────────┐
          ▼            ▼            ▼
┌──────────────────┐ ┌──────────────┐ ┌──────────────────┐
│  ChromaDB        │ │  HuggingFace │ │   Groq Cloud     │
│  (data/vectordb) │ │  Embeddings  │ │  (LLM inference) │
│  Vector Store    │ │ all-MiniLM   │ │ llama-3.1-8b     │
└──────────────────┘ └──────────────┘ └──────────────────┘
```

Layer	Technology
Frontend	React 18 + Vite, Vanilla CSS
Backend	FastAPI + Uvicorn
Embeddings	`all-MiniLM-L6-v2` (HuggingFace, runs locally)
Vector Store	ChromaDB (persisted to `data/vectordb/`)
LLM Inference	Groq Cloud — `llama-3.1-8b-instant` (fast) / `llama-3.3-70b-versatile` (smart)
PDF Processing	pdf.js (runs entirely in-browser — no transcript ever leaves the client)

Application Workflow

1. **Local Anonymization**: The student uploads their Purdue transcript PDF in the "Anonymize" tab. The browser extracts the text and strips out personally identifiable information (PII) like names, emails, and phone numbers locally.
2. **Context Injection**: The anonymized transcript is loaded into the chat interface.
3. **Retrieval (RAG)**: When the student asks about their CODO eligibility, the FastAPI backend converts their query into a vector and searches the local ChromaDB for the most relevant official CODO policy documents.
4. **LLM Generation**: The backend constructs a highly specific prompt containing:
   - The retrieved CODO policy rules.
   - The student's anonymized transcript data.
   - Strict instructions to evaluate what requirements are met, what are missing, and calculate an overall "Eligible" or "Not Eligible" status.
5. **Response**: The Groq hardware accelerates the LLM inference, instantly returning the custom analysis and actionable next steps back to the student's chat UI.

Prerequisites

- **Python 3.10+**
- **Node.js 18+** and **npm**
- **Git**
- A **Groq API key** — get one free at console.groq.com

Setup

1. Clone the Repository

```
git clone https://github.com/sanilshetty/Academic-Advising-S26.git
cd Academic-Advising-S26
```

2. Create and Activate a Python Virtual Environment

```
# Create the environment
python -m venv .venv

# Activate it (Windows PowerShell)
.venv\Scripts\Activate.ps1

# Activate it (Windows CMD)
.venv\Scripts\activate.bat

# Activate it (macOS/Linux)
source .venv/bin/activate
```

3. Install Python Dependencies

```
pip install -r requirements.txt
```

4. Configure Environment Variables

Create a `.env` file in the project root:

```
GROQ_API_KEY=your_groq_api_key_here
```

5. Install Frontend Dependencies

```
cd frontend
npm install
cd ..
```

Building the Knowledge Base

Before running the app, ingest the CODO policy documents into the vector database.

1. Add Documents

When you clone the repository, the `data/raw_data/` and `data/vectordb/` directories are completely empty (except for `.gitkeep` tracker files) to prevent developers' local database experiments from clashing on GitHub.

To populate your local knowledge base:

1. Obtain the official Purdue CODO policy source documents (PDFs or `.txt` files).
2. Place these documents into the empty `data/raw_data/` directory.

Note: Logan created a doc in the google drive folder titled CS CODO Requirements Knowledge Base. If you copy and paste that into a text file and place it in the data/raw_data/ folder, it will be used as the knowledge base. Feel free to tweak the knowledge base to get more accurate reponses.

You can also use the knowledge base management script:

```
# Add a file and immediately re-ingest
python manage_kb.py add path/to/document.pdf --ingest

# List documents currently in the knowledge base
python manage_kb.py list
```

```
# Remove a document and re-ingest
python manage_kb.py remove document.pdf --ingest
```

2. Run Ingestion

```
# First-time ingestion
python ingest.py

# Reset the database and re-ingest all documents
python ingest.py --reset
```

A `data/vectordb/` directory will be created containing the persisted ChromaDB vector store.

Running the Application

You need **two terminals** running simultaneously.

Terminal 1 — FastAPI Backend:

```
uvicorn server:app --reload --port 8000
```

Terminal 2 — React Frontend:

```
cd frontend
npm run dev
```

Open your browser to `http://localhost:5173`.

Using the App

1. **Anonymize Tab**: Upload your Purdue transcript PDF and enter your full name. The app uses pdf.js to extract the text entirely in your browser, then strips your name, email, and phone number before anything is sent to the server.
2. **Chat Tab**: Once your transcript is anonymized, it is automatically loaded into the chat context. Ask the AI advisor about your CODO eligibility. The sidebar shows the live status of the vector database and lets you upload additional knowledge base documents without restarting the server.

Evaluation (Optional)

To evaluate RAG pipeline quality using RAGAS metrics (faithfulness, answer relevancy):

```
python evaluate.py
```

Results are printed to the console and saved to `evaluation_report.json`.

Project Structure

```
Purdue-CS-CODO-RAG/
├── server.py               # FastAPI backend
├── rag_pipeline.py         # RAG chain (retriever + Groq LLM)
├── ingest.py               # Document ingestion → ChromaDB
├── manage_kb.py            # CLI tool to add/remove/list KB docs
├── evaluate.py             # RAGAS evaluation script
├── requirements.txt        # Python dependencies
│
├── frontend/               # React + Vite frontend
│   ├── index.html          # HTML entry point
│   ├── vite.config.js      # Vite dev server configuration
│   ├── package.json        # Node dependencies
│   └── src/
│       ├── main.jsx        # React app entry point
│       ├── App.jsx         # Root component — tab routing
│       ├── api.js          # Fetch wrappers for the FastAPI backend
│       ├── index.css       # Global design system / all styles
│       └── components/
│           ├── Anonymizer.jsx  # PDF redaction panel (Tab 1)
│           ├── ChatPanel.jsx   # Chat interface (Tab 2)
│           └── Sidebar.jsx     # DB status + KB upload
│
├── data/
│   ├── raw_data/           # Source documents (PDFs, TXTs) — add yours here
│   └── vectordb/           # Persisted ChromaDB vector store (auto-generated)
│
└── tools/                  # Debug and inspection utilities
```

Tuning the Model

If you'd like to tune the RAG pipeline or experiment with different prompts, embeddings, or parameters, here is a guide on exactly where to make those changes:

1. System Prompts

Files: `rag_pipeline.py` and `system_prompt.txt` The AI's personality and the instructions for generating an answer based on context are easily editable.

- **Contextualization Prompt (`rag_pipeline.py` Line 99):** This dictates how the bot rewrites a follow-up question (e.g. "What about for Computer Engineering?") into a standalone question (using chat history) before querying the database.

- **Generation System Prompt (`system_prompt.txt`):** This is the master prompt. It casts the LLM as the "Purdue Undergraduate Academic Advising Assistant" and establishes tone rules. Modify this file to completely change how the bot shapes its final answer.

> **Note:** Because `system_prompt.txt` is read once on startup and Uvicorn's `--reload` flag only watches python files by default, you must **manually restart the FastAPI backend process** for your prompt changes to take effect.

2. Changing the Knowledge Base

You can add new documents to the knowledge base by adding them to the `data/raw_data/` directory and running the `ingest.py` script. You can also change files and re-ingest to see how the model's responses change.

3. Vector Database Chunking Strategy

File: `ingest.py` When documents are parsed, they are split into overlapping chunks for vectorization. Smaller chunks provide more dense semantic matching, while larger chunks provide broader context to the LLM.

- **Line 15 & 16:** `CHUNK_SIZE = 1000` and `CHUNK_OVERLAP = 100` controls the text splitter.

4. Embedding Model

File: `ingest.py` AND `rag_pipeline.py` We currently use the fast, local `all-MiniLM-L6-v2` HuggingFace embedding model.

- **`ingest.py` (Line 19):** Change the `model_name` argument to use a different embedding model for generating the chroma database.
- **`rag_pipeline.py` (Line 68):** Change the `model_name` here as well so the retriever maps user questions to the same vector space.

5. LLM Selection (Groq Cloud)

File: `rag_pipeline.py` The pipeline is currently wired to Groq for near-instant inference.

- **Line 61 & 62:** Define the target Groq model strings. We default to `llama-3.1-8b-instant`.
- **Line 90:** The `ChatGroq()` class is instantiated here. You can swap this to OpenAI, Anthropic, or local Ollama classes if desired. You can also edit the `temperature` here.

6. Retrieval Search Strategy

File: `rag_pipeline.py` We retrieve a set of relevant document chunks before generation. We currently use Maximal Marginal Relevance (MMR) search instead of standard similarity search to enforce diversity in the retrieved chunks.

- **Line 72-74:** The base retriever is defined. You can change `search_type="mmr"` back to standard `"similarity"`.
- **Line 74:** `{"k": 5, "fetch_k": 20}` means it fetches 20 documents via similarity search, then uses MMR to narrow it down to the 5 most diverse chunks to pass to the context window.

File Descriptions

Backend

File	Role
`server.py`	FastAPI application. Exposes three endpoints: `GET /status` (checks if the vector DB exists), `POST /ingest` (accepts PDF/TXT uploads, saves them to `data/raw_data/`, and rebuilds the vector database), and `POST /chat` (runs a single RAG turn, accepts the full chat history for multi-turn context). Warms up the RAG chain on startup to minimize first-request latency.
`rag_pipeline.py`	Core RAG logic. `get_retriever()` loads ChromaDB with `all-MiniLM-L6-v2` embeddings and configures MMR search (k=5, fetch_k=20) for diverse document retrieval. `get_rag_chain()` builds a two-stage LangChain pipeline: (1) a **contextualize chain** that rewrites follow-up questions into standalone queries using chat history, then (2) a **QA chain** that injects the retrieved policy context and the student's anonymized transcript into a system prompt and calls Groq's `llama-3.1-8b-instant`. Both chains use `ChatGroq` and log retrieval debug info to the console.
`ingest.py`	Document ingestion pipeline. Reads all PDFs and `.txt` files from `data/raw_data/`, splits them into overlapping chunks using `RecursiveCharacterTextSplitter`, generates embeddings with `all-MiniLM-L6-v2`, and persists the resulting vectors to `data/vectordb/` via ChromaDB. Supports a `--reset` flag to wipe and rebuild the database from scratch.
`manage_kb.py`	CLI knowledge base manager. Supports `add`, `remove`, and `list` sub-commands to manage files in `data/raw_data/`. The `--ingest` flag triggers `ingest.py` automatically after any change, making it easy to update the knowledge base without manually running the ingestion script.
`evaluate.py`	RAGAS evaluation harness. Runs a set of predefined test questions through the RAG chain, collects the answers and retrieved context, then reports faithfulness and answer relevancy scores. Results are saved to `evaluation_report.json`.

Frontend

File	Role
`frontend/src/App.jsx`	Root React component. Manages two pieces of global state: the active tab (`anonymize` or `chat`) and `transcriptData` (the redacted transcript string). When `Anonymizer` completes a redaction, it calls `onRedactionComplete`, which stores the cleaned text in state and auto-switches to the Chat tab after 800 ms so the user sees the success banner.

File	Role
`frontend/src/api.js`	Thin fetch wrapper for the FastAPI backend. Exports three functions: `getStatus()` (polls `GET /status`), `ingestFiles(files)` (posts a `multipart/form-data` payload to `POST /ingest`), and `sendChat(question, transcriptData, history)` (posts JSON to `POST /chat`). All functions throw descriptive errors on non-2xx responses.
`frontend/src/index.css`	Global design system. Defines all CSS custom properties (colors, spacing, radii, shadows), the two-column `layout` grid, sidebar styles, tab bar, chat bubble styles, the anonymizer panel, form inputs, the drag-and-drop PDF zone, stat cards, and micro-animations (spinner, typing dots). No CSS framework — all styles are hand-authored vanilla CSS.
`frontend/src/components/Anonymizer.jsx`	Tab 1 — PDF redaction panel. Lazily loads `pdf.js` from CDN, extracts text from every page of the uploaded transcript, then runs a `redactPII()` function that strips the student's full name (case-insensitive), email addresses, and phone numbers using regex, replacing each match with `[REDACTED]`. All processing is fully client-side — no data is sent to the server. Displays a stats strip (pages, redactions, word count) and a read-only textarea with the redacted output. On completion, fires `onRedactionComplete` to pass the clean text up to `App`.
`frontend/src/components/ChatPanel.jsx`	Tab 2 — conversational chat interface. Maintains the full message history in local state and passes it to `api.js` on each send so the backend can contextualize follow-up questions. Auto-scrolls to the latest message, auto-resizes the textarea, and shows an animated three-dot typing indicator while awaiting a response. Displays Groq inference latency (in seconds) beneath each assistant reply. If the anonymized transcript is loaded, a 🔒 badge is shown at the top of the panel.
`frontend/src/components/Sidebar.jsx`	Persistent left sidebar. Polls `GET /status` every 5 seconds to display a live green/red indicator for the vector database. Also provides a knowledge base uploader: the user selects one or more PDF/TXT files and clicks ⚡ **Ingest Files**, which calls `POST /ingest` and displays a success or error message. The sidebar is always visible regardless of which tab is active.